

TCEQ Programmer IV — Interview Preparation Guide

Role: Programmer IV (State Job Code 0244A, Grade B26)
Division: Information Resources Division (IRD), Administrative Services
Location: 12100 Park 35 Circle, Austin, TX 78753
Salary: \$7,745/month (~\$92,940 annual), fixed
Openings: 2 | **Schedule:** M–F, 8am–5pm | **Overtime:** Exempt
Posted: Jan 16 – Jan 30, 2026

Table of Contents

1. [How the Scoring Works \(Exploit the Matrix\)](#)
 2. [The Job in Six Bullets](#)
 3. [Interview Format & Logistics](#)
 4. [JAD Facilitation & Strategic Advisory](#)
 5. [Technical Design & Architecture](#)
 6. [Code Leadership & Best Practices](#)
 7. [QA/QC & Testing Coordination](#)
 8. [Customer Support](#)
 9. [Behavioral Question Bank](#)
 10. [Written Test / Practical Exercise Prep](#)
 11. [Compensation Positioning Strategy](#)
 12. [Day-Of Checklist](#)
 13. [Technical Question Bank — By Stack](#)
 14. [Senior-Level Behavioral Questions](#)
 15. [Questions You Should Ask](#)
 16. [On-Site Interview Etiquette](#)
 17. [System Design Questions](#)
 18. [Algorithm & Data Structure Questions](#)
 19. [Advanced Java Questions](#)
 20. [Web Architecture Questions](#)
-

1. How the Scoring Works

***This is the single most important thing to understand.** Texas state hiring panels use a standardized Interview Scoring Matrix. They cannot make "gut feeling" decisions.*

How It Works

- Each answer is scored on a **Likert scale (1–5)** by every panelist independently
- Scores are aggregated → highest cumulative score wins the recommendation to HRSS
- The panel **can only award points for things you explicitly say** — if you don't say the keyword, they can't check the box

Drop These FJD Keywords Into Every Answer

The posting uses specific language. Mirror it:

FJD Phrase (verbatim from posting)	Work it in naturally
"agency information technology (IT) standards"	"...adhering to agency IT standards..."
"requirements documentation"	"...based on requirements documentation..."
"requirements and design documentation"	"...tested against requirements and design documentation..."
"configuration management"	"...tracked through our configuration management process..."
"quality assurance / quality control"	"...coordinated QA/QC testing..."
"customer acceptance testing"	"...guided the customer through customer acceptance testing..."
"peer reviews"	"...submitted for peer review before merging..."
"coding best practices, techniques, and training needs"	"...reviewing coding best practices and identifying training needs..."
"charters" / "business cases"	"...documented in the project charter and business case..."
"project plans and status reports"	"...communicated progress through status reports..."
"sponsor meetings"	"...presented at sponsor meetings..."
"facilitate joint application development (JAD) sessions"	"...which we defined during the JAD session..."
"analysis, development, programming, and implementation/modification"	Use this full sequence when describing your work

Rule of thumb: If you weave 3–4 FJD phrases into each answer, you're maximizing your matrix score.

2. The Job in Six Bullets

The posting lists exactly six responsibilities. Expect questions mapped to each:

#	Responsibility (from posting)	What They'll Ask About
1	Lead/participate in analysis, development, programming, implementation/modification	"Tell us about a system you led from analysis to deployment"
2	Lead coding tasks; review/evaluate coding best practices and training needs	"How do you enforce coding standards?" / "How do you mentor?"
3	Provide technical advice and design tasks based on requirements documentation	"Walk us through a design decision" / "How do you translate requirements into architecture?"
4	Coordinate QA/QC; participate/lead peer reviews; guide customer acceptance testing	"How do you define 'done'?" / "How do you guide non-technical users through testing?"
5	Create charters, business cases, project plans, status reports; facilitate JAD sessions	"How do you facilitate requirements gathering?" / "How do you manage stakeholder conflict?"
6	Provide customer support to agency staff	"How do you handle a frustrated user?"

Three Key Qualifications (from posting)

These are what the scoring matrix will weight most heavily:

1. **ColdFusion, Java, Hibernate, Oracle, SQL, PL/SQL, JavaScript, CSS, XML, HTML** — your demo project covers all of these
2. **Configuration management, quality assurance, change management** — discuss branching strategy, CI gates, peer review, and release checklists
3. **Business analysis: requirements, design/test plan documentation, JAD sessions** — this is the differentiator

3. Interview Format & Logistics

What to Expect	Details
Panel size	2–3: hiring manager + technical SME + possibly business unit rep
Style	Scripted questions read in order; panelists will not rephrase or go off-script
Multi-part questions	They may only repeat the entire question verbatim, not break it down
Duration	~1 hour; possibly phone screen + in-person/Teams
Written component	Likely — SQL debugging, code review, design prompt, or requirements writing
Timeline to offer	4–6 weeks from interview
Background check	Criminal history records search confirmed in posting
Critical infrastructure	This role may touch critical infrastructure (cybersecurity, hazardous waste systems, water treatment) per Texas Business & Commerce Code §117.001(2)

Multi-Part Question Strategy

1. **Jot key phrases** on your notepad as they read
2. **Restate:** "That question has three parts. Let me address each."
3. **Label explicitly:** "For part one... For part two..."
4. **Summarize:** "To bring it all together..."

4. JAD Facilitation & Strategic Advisory

The posting explicitly calls out: "facilitate joint application development (JAD) sessions." This is the differentiator between a mid-level coder and a Programmer IV.

What They'll Ask

"How do you facilitate a requirements gathering session?"

*"I use the JAD methodology. Before the session, I prepare an agenda, identify the right stakeholders — program area SMEs, IT security, the development lead — and distribute pre-read materials. During the session, I act as **facilitator**: I enforce the agenda, translate technical jargon into business language, and maintain a **parking lot** for off-scope ideas. I assign a scribe to capture*

decisions in real time. Outputs include **requirements documentation** with acceptance criteria, process flow diagrams, and a **project charter** with scope and constraints — all ready for **design** review and eventually driving **customer acceptance testing** scenarios."

Conflict Scenarios

Scenario	Your Response
"A program manager demands a feature that violates IT security."	"I validate their underlying business need, then present the constraint factually. I collaboratively design a secure alternative that achieves the same outcome, adhering to agency IT standards ."
"Participants derail into unrelated complaints."	"I acknowledge: 'That's important — let me add it to the parking lot .' Then redirect: 'For this session, we need to finalize the permit workflow by 3 PM.'"
"Two stakeholders disagree on a business rule."	"I restate each position neutrally, ask: 'Can we test both against the requirements documentation ?' If unresolvable, I document both options with trade-offs and escalate to the sponsor meeting ."

Your STAR Story for JAD / Advisory

Situation	Building the TCEQ compliance dashboard required defining what metrics matter — facilities, violations, penalties, overdue inspections, expiring permits — across multiple stakeholder perspectives.
Task	Translate regulatory compliance requirements into a coherent data model and dual-view UI.
Action	I created requirements documentation defining 9 dashboard metrics with clear acceptance criteria. I modeled the domain: Facilities → Permits, Inspections, Violations (1:N). I documented the business case for dual rendering (interactive SPA + server-rendered report) to serve different user needs. The compliance scoring algorithm was specified in design documentation before coding began.
Result	9 real-time metrics, 3 chart visualizations, searchable facility table — all traceable back to the requirements documentation . Dual-view design demonstrated the flexibility to serve varied stakeholders.

5. Technical Design & Architecture

Posting says: "Provide technical advice and perform design tasks... based on requirements documentation while adhering to agency IT standards."

Architecture Talk Track

"Walk us through how you'd design a compliance tracking application."

"I'd use a **layered architecture** adhering to **agency IT standards**: presentation → controller → service → repository → persistence. In **Java/Spring**, the service layer owns business logic while the repository layer handles data access through **Hibernate/JPA** against **Oracle**. I'd design the Oracle schema with **sequences**, foreign keys with cascading deletes, **CHECK constraints**, and strategic indexes on all filter columns. The ColdFusion presentation layer could coexist by pointing to the same Oracle datasource during a

phased migration. For security, all queries use parameterized inputs to prevent **SQL injection**, and all output is encoded to prevent XSS."

Database (Oracle SQL / PL/SQL)

Q: "Walk me through diagnosing a slow query."

"First, I pull the execution plan — `EXPLAIN PLAN FOR` + `DBMS_XPLAN.DISPLAY` — looking for full table scans where there should be index scans. I check for stale statistics with `SELECT last_analyzed FROM user_tables` and regather with `DBMS_STATS`. Common fixes: add missing indexes (especially on foreign key columns — in my demo I index every FK), rewrite correlated subqueries as JOINS, and check for implicit type conversions. In my compliance demo, I created indexes on `FACILITY_ID`, `STATUS`, `SEVERITY`, and `EPA_ID` because those are the dashboard's filter columns."

Q: "How do you design backwards-compatible schema changes?"

"Three rules: (1) **additive only** during deployment — add columns as nullable or with defaults; (2) **schema first, then code** — so both old and new versions work; (3) **versioned migrations** with rollback scripts. Every change goes through **configuration management** and **peer review**. This aligns with TCEQ's **change management** expectations."

Web Services (SOAP vs REST)

Q: "When would you pick SOAP over REST?"

"**SOAP** for formal inter-agency data exchanges where you need WS-Security for signed/encrypted messages, WSDL contracts, and transaction support. Government-to-government integrations often mandate SOAP for its audit trail. **REST** for internal APIs consumed by JavaScript frontends where JSON is natural. My demo uses REST (`/api/facilities`, `/api/dashboard/stats`) for the internal dashboard."

Java + Hibernate

Q: "Explain a production issue caused by ORM behavior."

"The N+1 problem: fetching a list of facilities then iterating to access `facility.getPermits()` fires one SELECT per facility. In my demo, `getFacilitySummaries()` does this — fine on H2 in-memory but devastating on production **Oracle** with thousands of rows. Fix: `JOIN FETCH` in JPQL or `@EntityGraph`. I'd catch this during **QA/QC** by monitoring query counts in integration tests and flagging during **peer review**."

ColdFusion

Q: "Show me a ColdFusion pattern you'd use."

"In my `facility_search.cfm`: dynamic `WHERE 1=1` with conditional `<cfif>` blocks, `<cfqueryparam>` with explicit `cfsqltype` for SQL injection prevention, `HTMLEditFormat()` for XSS protection, and server-side pagination with

`startrow / maxrows` . In `permit_report.cfm` : `<cfstoredproc>` for calling Oracle **PL/SQL**, `<cfchart>` for visualization, **Query-of-Queries** for in-memory reshaping, and `<cfdocument>` for PDF generation."

Linux / Shell Scripting

Q: "How do you approach production shell scripts?"

"Three principles: (1) **idempotent** — `mkdir -p` , `CREATE TABLE IF NOT EXISTS` ; (2) **fail-fast** — `set -euo pipefail` ; (3) **logged** — redirect output to timestamped log files. Secrets come from environment variables, never hardcoded."

Change Management

Q: "How do you ship safely?"

"Feature branch → **peer review** → CI runs tests → deploy to staging → **QA/QC** sign-off against **design documentation** → **change request** approved → production deploy with rollback script. My demo has `./mvnw test` as a CI gate — all 15 tests must pass. In production, I'd add static analysis and schema migration validation."

6. Code Leadership & Best Practices

Posting says: "Lead coding tasks... Review and evaluate coding best practices, techniques, and training needs."

STAR Story

Situation	Needed to build a compliance system demonstrating production patterns across the full TCEQ tech stack: Java, Hibernate, Oracle, SQL, PL/SQL, ColdFusion, JavaScript, CSS, XML, HTML .
Task	Structure it so any developer — including junior Programmers I–III — could maintain and extend it.
Action	Enforced layered architecture : model → repository → service → controller. Every entity is documented. Used both annotation and XML Hibernate mappings to demonstrate legacy and modern approaches. Wrote 15 JUnit 5 tests covering repository queries, service logic, and compliance scoring. Oracle-compatible DDL uses consistent naming (<code>FK_</code> , <code>UK_</code> , <code>CHK_</code> , <code>IDX_</code> prefixes).
Result	Self-documenting codebase: <code>architecture.md</code> maps every requirement to its implementation. Tests pass in one command. A new developer onboards by reading the README, which models the kind of training artifact I'd create for the team.

Q: "How do you handle code reviews?"

"I review four dimensions: (1) **correctness** — does it match **requirements documentation**?; (2) **security** — parameterized queries, input validation, output encoding; (3) **maintainability** — clean naming, separation of concerns; (4) **test coverage** — untested logic triggers a revision. I give constructive, specific feedback and treat **peer reviews** as **training** opportunities for junior staff — which the posting explicitly calls out as 'evaluating training needs.'"

7. QA/QC & Testing Coordination

Posting says: "Coordinate testing and Quality Assurance/Quality Control tasks... participate/lead peer reviews. Guide and assist customers with customer acceptance testing."

STAR Story

Situation	The compliance scoring algorithm is business-critical — wrong scores misrepresent a facility's environmental status.
Task	Ensure scoring logic was thoroughly tested, traceable to requirements and design documentation .
Action	Wrote JUnit tests for boundary conditions: clean facility (≥ 90), facility with 2 CRITICAL violations (≤ 40). Verified deduction table: -5 MINOR / -15 MAJOR / -30 CRITICAL / -10 expired permit / -5 overdue inspection. Documented the algorithm in <code>architecture.md</code> so QA/QC reviewers and customer acceptance testing participants could verify intent without reading code.
Result	All 15 tests pass. Scoring rules are documented, testable, and traceable. Any auditor can verify coverage.

Q: "How do you guide customers through acceptance testing?"

*"I create a **test plan** derived from acceptance criteria defined during the **JAD session**. Steps are numbered in plain language: 'Step 1: Search for facility X. Expected: 3 results appear.' I walk the user through the first scenarios live, then observe. Defects are tracked with severity tags, triaged, re-tested after fixes. The goal is making the business user feel ownership over quality — not just rubber-stamping."*

8. Customer Support

Posting says: "Provide customer support to agency staff specific to agency applications, IT policies, and general related IT areas."

Q: "How do you support agency staff?"

*"With patience and empathy. Agency staff are domain experts — environmental scientists, permit reviewers — not IT professionals. I **listen** to understand their workflow before diagnosing the technical root cause. I explain fixes in business-process terms, not stack traces. In my application design, I build defensively: REST APIs return proper HTTP 404s for missing records instead of 500s, ColdFusion templates use `<cftry>/<cfcatch>`, and the service layer uses `@Transactional(readOnly = true)` to prevent accidental mutations."*

9. Behavioral Question Bank

"Why should we hire you?"

"I bring the exact stack this posting lists — **ColdFusion, Java, Hibernate, Oracle, SQL, PL/SQL, JavaScript, CSS, XML, HTML** — demonstrated in a working compliance tracking application. But this role isn't just about coding. It explicitly requires **facilitating JAD sessions**, creating **charters and business cases**, **leading peer reviews**, and guiding **customer acceptance testing**. I have experience bridging business requirements and technical execution across that full lifecycle. I'm motivated by TCEQ's mission of protecting public health and natural resources, and I value the **long-term stability** and meaningful impact this work provides."

(Hits: full tech stack, JAD, charters, peer reviews, customer acceptance testing, mission, retention signal)

"Tell me about a conflict with a coworker."

"I disagreed with a teammate about refactoring vs. patching a legacy module. Rather than argue from instinct, I proposed we **timebox a spike** — two hours to estimate scope. The spike showed a 3-day refactor with a clear path, so we agreed on it with a shim to unblock the sprint. Takeaway: propose a data-driven evaluation. This is the same approach I'd use resolving disagreements in a **JAD session** — de-escalate, gather evidence, decide collaboratively."

"Name a time you failed."

"I deployed a schema migration that passed QA on small test data but locked a production table for 20 minutes on full volume. I learned: (1) test migrations against production-scale data; (2) add migration timing to **QA/QC**; (3) include rollback scripts in every **change management** request. I now treat schema changes with the same rigor as code — **peer review**, staging validation, documented rollback."

"How do you explain technical concepts to non-technical users?"

"Analogies from their domain. 'A database index is like filing cabinet tabs — without them you open every drawer to find Harris County.' In my demo I built dual views — interactive SPA and traditional server-rendered page — because different users have different preferences. During **JAD sessions**, I translate developer jargon into business-process language in real time."

"How do you prioritize competing demands?"

"Impact × urgency. A production bug blocking compliance reporting trumps a feature request. I communicate reasoning through **status reports** so stakeholders understand. If priorities conflict at leadership level, I escalate to the **project sponsor** with a clear trade-off write-up."

10. Written Test / Practical Exercise Prep

SQL Exercise

"Find the bug and fix it."

```
-- Buggy: double LEFT JOIN fans out rows
SELECT f.facility_name,
       COUNT(p.id) AS permit_count,
       COUNT(v.id) AS violation_count
FROM   facilities f
LEFT JOIN permits    p ON f.id = p.facility_id
LEFT JOIN violations v ON f.id = v.facility_id
GROUP BY f.facility_name;
```

Answer: "Double JOIN creates a Cartesian product — 3 permits × 2 violations = 6 rows, inflating both counts. Fix: `COUNT(DISTINCT p.id)` and `COUNT(DISTINCT v.id)`, or restructure as correlated subqueries."

Code Review

"Identify issues."

```
public List<Facility> search(String term) {
    String sql = "SELECT * FROM facilities WHERE name LIKE '%" + term + "%'";
    return jdbcTemplate.query(sql, new FacilityRowMapper());
}
```

Answer: "(1) **SQL injection** — string concatenation; use `?` placeholders. (2) **No null check** on `term`. (3) **Leading wildcard** prevents index use — consider Oracle Text `CONTAINS()`. (4) **No case normalization** — add `UPPER()`. (5) **No pagination** — unbounded `SELECT *`."

Design / Requirements Prompt

"A division needs permit renewal tracking with notifications. Outline your approach."

Answer skeleton:

- **Charter:** Scope, stakeholders, constraints, timeline
- **Requirements (from JAD session):** Track expiry dates, auto-notify at 90/60/30 days, capture renewal submissions, approval workflow
- **Data model:** `PERMIT_RENEWALS (id, permit_id FK, due_date, notification_date, status, reviewer_id)`
- **Architecture:** Scheduled batch job (Spring `@Scheduled` or Oracle `DBMS_SCHEDULER`) → queries expiring permits → creates notification records → SMTP integration
- **QA/QC:** Unit tests for date logic, integration test with mock email, **customer acceptance testing** script
- **Change management:** Feature branch → **peer review** → staging → QA sign-off → change request → production with rollback

11. Compensation Positioning Strategy

Two openings, fixed salary at \$7,745/month. The panel wants to know you'll stay.

What NOT to Say

- Don't ask about rapid salary advancement or equity packages
- Don't compare to private sector offers
- Don't express urgency about promotion timelines

What TO Say

- **"I value long-term stability"** — signals retention intent
- **"I'm motivated by TCEQ's mission — protecting public health and natural resources"** — mirrors the posting's own language
- **"The total compensation is attractive — 401(k)/457, full health/dental/vision, longevity pay, wellness programs"** — shows you've read the benefits section
- **"I want to build and maintain systems that serve Texans for years"** — positions you as a long-term investment

Benefits (from the actual posting)

- 401(k) and 457 plans
 - Health, Vision, Dental insurance + optional FSA
 - Paid holidays (national + state)
 - Professional development opportunities
 - Longevity pay (increases with years of state service)
 - Wellness Program + onsite Nurse Practitioner at HQ
 - Work-Life Balance
-

12. Day-Of Checklist

Before

- ☐ Run `./mvnw clean test` — all 15 tests pass
- ☐ Start app: `./mvnw spring-boot:run` — dashboard loads at `localhost:8080`
- ☐ Print: resume + `docs/architecture.md` diagram
- ☐ Practice the STAR stories out loud (under 2 minutes each)
- ☐ Review: ColdFusion templates, PL/SQL procedures, `schema.sql`
- ☐ Re-read this guide, especially **Section 1** (scoring), **Section 4** (JAD), and the **keyword list** below

Bring

- ☐ Notepad and pen (for multi-part questions)
- ☐ Printed portfolio materials
- ☐ Photo ID (building access at 12100 Park 35 Circle)
- ☐ Documentation for criminal history records check if requested

Questions to Ask Them

1. "What does the **ColdFusion-to-Java migration** roadmap look like?"
2. "How does the team structure **JAD sessions** — formal workshops or shorter sprint-style?"
3. "What does the **peer review and change management** process look like here?"
4. "Are there opportunities to improve **automated testing or CI/CD**?"
5. "What's the current **telework/hybrid policy** for the IRD?"
6. "What systems or applications would I be working with day one?"

Logistics

- ☐ Business attire
 - ☐ Arrive 15 minutes early at **12100 Park 35 Circle, Austin 78753**
 - ☐ Plan parking
 - ☐ If virtual (Teams): test camera + mic, clean background
-

Quick Reference: FJD Keywords to Deploy

These are **verbatim from the posting**. Say them out loud during the interview:

analysis, development, programming, implementation/modification · agency IT standards · coding best practices, techniques, and training needs · requirements documentation · design documentation · configuration management · quality assurance / quality control · peer reviews · customer acceptance testing · charters · business cases · project plans and status reports · sponsor meetings · JAD sessions · customer support · application maintenance, support, and development

Every keyword you say is a box the panel can check on their scoring matrix. **Say them.**

Critical Infrastructure Note

The posting includes: "Employees in this classification series may research, work on, or have access to critical infrastructure, including but not limited to a communication infrastructure system, cybersecurity system, electric grid, hazardous waste treatment system, or water treatment facility."

This means: (1) expect a thorough background check, (2) emphasize **security-first coding practices** in your answers, and (3) demonstrate awareness that the software you build has real-world environmental and public health consequences.

13. Technical Question Bank — By Stack

Each technology listed in the posting could generate 3–5 interview questions. Prepare answers for all of them.

ColdFusion

#	Question	Key Points for Your Answer
1	"What is <code><cfqueryparam></code> and why is it important?"	Prevents SQL injection by binding parameters with typed values (<code>cfsqltype</code>). Never concatenate user input into <code><cfquery></code> .
2	"Explain Query-of-Queries (QoQ). When would you use it?"	In-memory SQL against an existing ColdFusion recordset — useful for reshaping, filtering, or joining cached data without hitting the DB again. My <code>permit_report.cfm</code> uses QoQ to filter violations by severity from an already-fetched dataset.
3	"How do you generate PDF reports in ColdFusion?"	<code><cfdocument format="PDF"></code> wraps HTML/CSS content and renders it as a downloadable PDF. Supports headers, footers, page breaks. Useful for compliance reports that agency staff need to print or archive.
4	"How do you call a stored procedure from ColdFusion?"	<code><cfstoredproc procedure="proc_name" datasource="ds"></code> with <code><cfprocparam></code> for IN/OUT parameters and <code><cfprocresult></code> to capture result sets. My <code>permit_report.cfm</code> demonstrates this pattern with Oracle PL/SQL.
5	"How would you migrate a ColdFusion application to Java?"	Incremental: (1) identify the ColdFusion pages with the most business logic; (2) extract that logic into Java services called via REST; (3) ColdFusion pages become thin wrappers calling the Java API; (4) eventually replace ColdFusion templates with Thymeleaf or a JS frontend; (5) both layers share the same Oracle datasource during transition.
6	"What is the ColdFusion Application scope vs Session scope?"	Application scope persists for the life of the application and is shared across all users (config, cached lookups). Session scope is per-user and expires after timeout. Overusing Application scope with mutable data causes concurrency bugs.

Java

#	Question	Key Points for Your Answer
1	"Explain the difference between an interface and an abstract class."	Interface = contract (what to do), abstract class = partial implementation (some behavior provided). Since Java 8, interfaces can have default methods, blurring the line — but abstract classes still allow state (instance fields) and constructors.
2	"What is dependency injection and why does it matter?"	Instead of a class creating its own dependencies (<code>new FacilityRepo()</code>), they're injected via constructor. Benefits: testability (inject mocks), loose coupling, easier configuration management . Spring Boot does this automatically with <code>@Autowired</code> / constructor injection.
3	"Explain <code>try-with-resources</code> ."	Automatically closes resources implementing <code>AutoCloseable</code> when the block exits — prevents resource leaks (DB connections, file handles). Critical in long-running agency applications.
4	"What are Java Streams? Give an example."	Functional-style pipeline for processing collections. In my demo: <code>violations.stream().filter(v -> "OPEN".equals(v.getStatus())).toList()</code> filters open violations without mutation. Readable, composable, parallelizable.
5	"How do you handle exceptions in a production Java application?"	Differentiate checked (recoverable, e.g., <code>IOException</code>) from unchecked (programming errors, e.g., <code>NullPointerException</code>). Use specific catches, log with context (facility ID, user, timestamp), return meaningful HTTP error codes from REST controllers (404, 400, 500), and never swallow exceptions silently.
6	"What is the Java Collections framework? When do you use a <code>Map</code> vs <code>List</code> ?"	<code>List</code> for ordered sequences (facility search results), <code>Map</code> for key-value lookups (dashboard stats by metric name). <code>LinkedHashMap</code> when insertion order matters (like my <code>getDashboardStats()</code> method — display order matches insert order).
7	"How does garbage collection work in Java?"	JVM automatically reclaims memory from objects with no references. Generational GC (Young Gen → Old Gen). Monitor with <code>-verbose:gc</code> . In long-running server apps, watch for memory leaks from unclosed resources, static collections that grow forever, or Hibernate session caches.

Hibernate / JPA

#	Question	Key Points for Your Answer
1	"What is the N+1 problem and how do you fix it?"	Fetching a parent list then lazily loading each child collection = 1 + N queries. Fix: <code>JOIN FETCH</code> in JPQL, <code>@EntityGraph</code> , or <code>@BatchSize</code> . My demo's <code>getFacilitySummaries()</code> is a textbook example — production fix would be a fetch join.
2	"Lazy vs Eager loading — when do you use each?"	Lazy (default for collections): loads on first access — good for large associations you may not need. Eager: loads immediately — use sparingly (small, always-needed associations). Eager on large collections causes massive joins.
3	"What is the Hibernate Session / EntityManager lifecycle?"	EntityManager is short-lived (per-request). Managed entities are tracked for dirty checking within a transaction. Detached entities (outside transaction) need <code>merge()</code> to reattach. In Spring, <code>@Transactional</code> manages this automatically.
4	"Explain the difference between JPA annotations and XML mapping."	Annotations (<code>@Entity</code> , <code>@Column</code>) are inline with the Java class — modern standard. XML (<code>Facility.hbm.xml</code>) separates mapping from code — legacy approach still used in older systems. My demo uses both to demonstrate proficiency with legacy and modern patterns.
5	"How do you handle database-generated IDs with Hibernate?"	<code>@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "facility_seq")</code> with <code>@SequenceGenerator</code> pointing to the Oracle sequence. My demo uses Oracle-style <code>CREATE SEQUENCE</code> and maps via annotations.
6	"What is dirty checking in Hibernate?"	Hibernate automatically detects field changes on managed entities and generates UPDATE statements at flush time. No explicit <code>save()</code> needed within a transaction. Can be a source of "phantom updates" if you modify entities unintentionally.

Oracle SQL

#	Question	Key Points for Your Answer
1	"Write a query to find facilities with more than 3 open violations."	<pre>SELECT f.facility_name, COUNT(v.id) AS viol_ct FROM facilities f JOIN violations v ON f.id = v.facility_id WHERE v.status = 'OPEN' GROUP BY f.facility_name HAVING COUNT(v.id) > 3;</pre>
2	"What is the difference between WHERE and HAVING?"	<code>WHERE</code> filters rows before grouping. <code>HAVING</code> filters groups after aggregation. You can't use aggregate functions in <code>WHERE</code> .
3	"Explain Oracle sequences vs auto-increment."	Oracle uses <code>CREATE SEQUENCE + NEXTVAL</code> for ID generation (explicit, controllable, gap-aware). MySQL/PostgreSQL use <code>AUTO_INCREMENT / SERIAL</code> . In Oracle, you can share sequences, set increment/cache values, and use them across triggers.
4	"What are the different types of JOINS? When would you use each?"	<code>INNER JOIN</code> : only matching rows. <code>LEFT JOIN</code> : all rows from left + matches from right (nulls if none). <code>FULL OUTER JOIN</code> : all rows from both sides. <code>CROSS JOIN</code> : Cartesian product (rarely wanted). For compliance: <code>LEFT JOIN</code> facilities to violations to include facilities with zero violations.
5	"How do you create an index and when should you?"	<code>CREATE INDEX idx_name ON table(column);</code> Use on: FK columns, columns in <code>WHERE/ORDER BY/GROUP BY</code> , columns used in <code>JOINS</code> . Don't over-index: each index slows <code>INSERT/UPDATE</code> . My demo indexes every FK and every filter column.
6	"Explain the difference between DELETE, TRUNCATE, and DROP."	<code>DELETE</code> : removes rows with optional <code>WHERE</code> , logged, can rollback, fires triggers. <code>TRUNCATE</code> : removes all rows, minimal logging, resets high-water mark, cannot rollback. <code>DROP</code> : removes the entire table structure. In a regulated environment, always prefer <code>DELETE</code> for audit trail.
7	"What is a CHECK constraint?"	<code>CHECK (status IN ('ACTIVE', 'INACTIVE', 'SUSPENDED', 'CLOSED'))</code> — enforces data validity at the database level, independent of application code. My schema uses <code>CHECK</code> constraints on every

		status/severity column.
--	--	-------------------------

PL/SQL

#	Question	Key Points for Your Answer
1	"Write a PL/SQL function that returns a compliance score."	Declare variables, SELECT INTO for counts, apply deduction formula, clamp to 0 with IF/THEN, RETURN. My <code>procedures.sql</code> has the complete Oracle PL/SQL version of <code>calculate_compliance_score</code> .
2	"What is the difference between a stored procedure and a function?"	Procedure: executes an action, no return value (uses OUT params). Function: returns a value and can be called in SQL statements. Use procedures for batch updates, functions for computed values (like scoring).
3	"What is NVL vs COALESCE ?"	<code>NVL(expr, default)</code> is Oracle-specific, evaluates both arguments. <code>COALESCE(expr1, expr2, ...)</code> is ANSI SQL, accepts multiple arguments, short-circuits. Prefer COALESCE for portability.
4	"How do you handle errors in PL/SQL?"	<code>EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;</code> — catch, clean up, re-raise. Use specific exception names when possible (<code>NO_DATA_FOUND</code> , <code>TOO_MANY_ROWS</code>). Never silently swallow exceptions in regulatory systems.
5	"What is a cursor? When do you use SYS_REFCURSOR ?"	Cursor = pointer to a result set. <code>SYS_REFCURSOR</code> is used as an OUT parameter to pass result sets back to Java/ColdFusion callers. My <code>get_overdue_inspections</code> procedure returns a <code>SYS_REFCURSOR</code> .

JavaScript

#	Question	Key Points for Your Answer
1	"Explain the difference between <code>var</code> , <code>let</code> , and <code>const</code> ."	<code>var</code> : function-scoped, hoisted (legacy). <code>let</code> : block-scoped, not hoisted. <code>const</code> : block-scoped, immutable binding (the reference can't change, but object contents can). Use <code>const</code> by default, <code>let</code> when reassignment is needed, avoid <code>var</code> .
2	"What is the Fetch API? How does it differ from XMLHttpRequest?"	Fetch returns Promises (cleaner async syntax with <code>.then()</code> or <code>async/await</code>). XHR is callback-based (older, verbose). My dashboard uses <code>fetch('/api/dashboard/stats').then(r => r.json())</code> to load data asynchronously.
3	"What is event delegation?"	Attach one event listener to a parent element instead of N listeners on child elements. When a child event bubbles up, check <code>event.target</code> to handle it. More efficient for dynamic lists (like facility search results).
4	"How do you prevent XSS in JavaScript?"	Never insert raw user input into the DOM via <code>innerHTML</code> . Use <code>textContent</code> for text, or sanitize with a library. On the server side, encode output (my ColdFusion uses <code>HTMLEditFormat()</code>).
5	"Explain closures."	A function retains access to its outer scope's variables even after the outer function has returned. Used for data privacy, callbacks, and factory patterns.
6	"What is <code>async</code> / <code>await</code> ?"	Syntactic sugar over Promises. <code>async</code> marks a function that returns a Promise; <code>await</code> pauses execution until the Promise resolves. Makes asynchronous code (API calls, DB queries) read like synchronous code.

CSS

#	Question	Key Points for Your Answer
1	"Explain the CSS box model."	Every element = content + padding + border + margin. <code>box-sizing: border-box</code> makes width include padding and border (predictable sizing). My dashboard CSS uses this for consistent layout.
2	"How do you make a layout responsive?"	Media queries (<code>@media (max-width: 768px)</code>), flexible units (<code>%</code> , <code>em</code> , <code>rem</code> , <code>vw</code>), Flexbox for 1D layouts, CSS Grid for 2D. Start with mobile-first, scale up.
3	"Flexbox vs Grid — when do you use each?"	Flexbox: one-dimensional (row OR column) — nav bars, card rows. Grid: two-dimensional (rows AND columns) — dashboard layouts, complex page structures.
4	"What is CSS specificity?"	Inline styles > IDs > classes/attributes > elements. <code>!important</code> overrides all (avoid it). Understanding specificity prevents "why isn't my style applying?" debugging sessions.
5	"How do you ensure accessibility in CSS?"	Sufficient color contrast (4.5:1 ratio), visible focus indicators (<code>:focus</code> styles), don't rely on color alone to convey meaning, use relative units (<code>rem</code>) for text sizing so users can scale, ensure touch targets are at least 44x44px.

XML / HTML

#	Question	Key Points for Your Answer
1	"What is the difference between XML and HTML?"	XML is a strict data format (case-sensitive, must close all tags, no predefined elements). HTML is a rendering language (forgiving, predefined elements like <code><h1></code> , <code><table></code>). In TCEQ: XML for data exchange (config, Hibernate mappings, web service payloads); HTML for user interfaces.
2	"What is semantic HTML and why does it matter?"	Use <code><header></code> , <code><nav></code> , <code><main></code> , <code><article></code> , <code><footer></code> instead of generic <code><div></code> . Screen readers and search engines understand semantic elements. Critical for Section 508 accessibility compliance.
3	"How do you validate an XML document?"	Against an XSD (XML Schema Definition) or DTD (Document Type Definition). Java provides <code>javax.xml.validation</code> for programmatic validation. Ensures data integrity in system integrations.
4	"What is XSLT?"	XML Stylesheet Language Transformations — transforms XML into other formats (HTML, text, another XML). Used in legacy reporting systems to convert data exports into readable reports.
5	"Explain HTML form validation."	Client-side: <code>required</code> , <code>pattern</code> , <code>type="email"</code> , <code>min</code> / <code>max</code> attributes. Server-side: always re-validate (never trust the client). My ColdFusion templates use <code><cfparam></code> for server-side defaults and <code><cfqueryparam></code> for database-level validation.

14. Senior-Level Behavioral Questions

Beyond the basics in Section 9, expect these deeper probes for a Programmer IV:

Leadership & Mentoring

#	Question	Approach
1	"How do you mentor a junior developer who is struggling with a technology?"	Be specific: pair programming sessions, assign progressively harder tasks, create documentation/runbooks, set up regular 1-on-1s. Tie to FJD: "evaluating training needs ."
2	"Tell me about a time you led a team through a difficult technical decision."	STAR format. Show consensus-building, data-driven evaluation, documentation of the decision rationale. Tie to " leading coding tasks."

3	"How do you handle a team member who consistently writes low-quality code?"	Private, constructive conversation first. Pair reviews. Check if it's a skill gap or a process gap (unclear standards? missing linting?). Escalate only if coaching fails. Avoid public shaming.
4	"How do you set and enforce coding standards across a team?"	Document standards in a team wiki, enforce via automated linting in CI, and reinforce through peer reviews . Lead by example — your own code follows the standards.

Project Management & Stakeholder Communication

#	Question	Approach
5	"Describe how you create a project plan for a new software initiative."	Start with the charter (scope, constraints, stakeholders). Break work into phases (analysis → design → develop → test → deploy). Create status reports at each milestone. Present at sponsor meetings .
6	"How do you manage scope creep?"	Define scope clearly in the charter and requirements documentation . When new requests come in, evaluate impact and present the trade-off: "We can add X, but Y will slip by two weeks. Here's the updated project plan ."
7	"Tell me about a time a project was behind schedule. What did you do?"	Be honest about the cause. Show how you communicated via status reports , re-prioritized features, and negotiated with stakeholders. Never hide bad news.
8	"How do you handle conflicting requirements from different stakeholders?"	Facilitate a structured discussion (as in a JAD session). Document each requirement, identify the conflict, present trade-offs with data, and escalate to the project sponsor for a final decision.

Adaptability & Problem-Solving

#	Question	Approach
9	"Describe a time you had to learn a new technology quickly."	Be specific about the technology, your learning method (documentation, proof-of-concept, mentorship from an SME), and the outcome. Tie to FJD: ability to "learn various programming languages."
10	"Tell me about a time you improved an existing process."	Could be CI/CD improvements, documentation standards, testing automation, or code review practices. Show measurement — "reduced deployment time from 4 hours to 30 minutes."
11	"How do you stay current with technology trends while maintaining legacy systems?"	Dedicated learning time, tech community participation, internal brown-bag sessions. Key: show you respect legacy systems while strategically modernizing.
12	"Describe a situation where you had to make a decision without complete information."	Show risk assessment, document assumptions, build in reversibility, communicate uncertainty to stakeholders.

Government/Regulatory Mindset

#	Question	Approach
13	"Why do you want to work in state government instead of the private sector?"	Emphasize: mission-driven work, long-term stability, building systems that serve millions, sustainable work-life balance. Never say "it's a fallback."

14	"How do you handle working in a highly regulated environment with strict processes?"	"I view governance as a quality framework, not an obstacle. In regulated environments, configuration management , peer reviews , and QA/QC gates aren't bureaucracy — they prevent production incidents that could have real environmental consequences."
15	"This role may involve maintaining legacy systems built decades ago. How do you feel about that?"	"Legacy systems often process the most critical data. I approach them with respect — understand the existing architecture before changing anything, add tests around the code I'm modifying, and modernize incrementally. My demo project demonstrates both legacy patterns (XML Hibernate mappings, ColdFusion) and modern approaches side by side."
16	"How do you handle a situation where you disagree with your manager's technical decision?"	"I present my case with data and a written comparison. If they still decide differently, I execute their decision fully and professionally. I document my concerns in the requirements documentation for future reference, but once the decision is made, I commit to it."

15. Questions You Should Ask

***You will be given time to ask questions.** This is not optional — asking zero questions signals disinterest. Prepare 4-5 from the list below and prioritize based on what comes up during the interview.*

About the Work

#	Question	Why You're Asking
1	"What applications or systems would I work on in my first 30/60/90 days?"	Shows you're thinking about ramp-up and immediate contribution.
2	"What's the current balance of new development vs. maintenance/support?"	Reveals the actual day-to-day workload.
3	"Is there an ongoing ColdFusion-to-Java (or other modernization) initiative?"	Shows you've read the tech stack and are thinking strategically.
4	"What does the deployment process look like — how does code move from dev to production?"	Reveals the change management maturity level.
5	"How does the team handle production emergencies or on-call?"	Clarifies after-hours expectations not obvious from the M-F 8–5 schedule.

About the Team

#	Question	Why You're Asking
6	"How large is the IRD development team, and how is it structured?"	Understanding team size, reporting lines, and whether there are sub-teams.
7	"What does the peer review process look like in practice?"	Shows FJD keyword awareness and genuine interest in their workflow.
8	"How does the team run JAD sessions — formal multi-day workshops or shorter, agile-style?"	Directly probes the #1 responsibility area (35% of the role).
9	"What professional development opportunities does the team take advantage of?"	Shows long-term growth mindset. Mirrors the posting's "Professional development opportunities" benefit.

About Policy & Logistics

#	Question	Why You're Asking
10	"What is the current telework/hybrid policy for the IRD?"	Ask this. State policies are shifting. Clarify before accepting an offer.
11	"Is the M–F 8–5 schedule fixed, or is there flexibility for the IRD?"	The posting says "Individual and work group schedule determined by IRD."
12	"What does the onboarding process look like for new Programmer IVs?"	Reveals how structured the ramp-up will be.
13	"Are there two separate teams hiring, or will both positions work on the same team?"	There are two openings — knowing the structure matters.

About Growth

#	Question	Why You're Asking
14	"What does career progression look like from Programmer IV within the IRD?"	Shows long-term intent (which they want to hear).
15	"Are there opportunities to contribute to architecture decisions or standards-setting?"	Signals that you want to operate at the senior level the role demands.

Tip: Listen carefully during the interview. If they mention a specific system (STEERS, CCEDS, etc.), ask a follow-up about it — "You mentioned STEERS — would I be working on that directly?"

16. On-Site Interview Etiquette

Before You Arrive

Do	Don't
Research the building location: 12100 Park 35 Circle, Austin 78753 (near Park 35 business park off I-35 North)	Don't rely on GPS alone — check for construction/detours in advance
Arrive 15 minutes early — clear security, find the right floor, use the restroom, settle your nerves	Don't arrive 30+ minutes early (it's awkward and puts pressure on staff)
Dress in business professional attire (suit/blazer, conservative colors) — state agencies lean formal	Don't dress business casual or wear tech company attire (hoodies, sneakers)
Bring 5 copies of your resume (one for each panelist + extras)	Don't bring only one copy
Bring a notepad and pen (for jotting multi-part questions)	Don't bring a laptop (it creates a barrier)
Turn your phone completely off (not silent — off)	Don't leave your phone on vibrate — state panels notice and it's scored negatively
Bring a portfolio folder or padfolio — professionally organized	Don't carry loose papers or a backpack if possible

When You Arrive

Do	Don't
Be polite to everyone — receptionist, security, anyone in the hall — panel members may ask front-desk staff for impressions	Don't be rude or dismissive to support staff
Present your photo ID at security if required (state buildings often have badge access)	Don't leave your ID in the car
Wait calmly in the lobby — review your prep notes quietly	Don't pace, make loud phone calls, or look frustrated
When escorted to the room, greet each panelist with a firm handshake, eye contact, and their name (if introduced)	Don't sit down before being invited

During the Interview

Do	Don't
Make eye contact with all panelists, not just the one asking the question	Don't fixate on only one person
When answering, address the panelist who asked , then sweep to others	Don't stare at the ceiling or table while thinking
Take brief notes when a multi-part question is read — the panel expects this	Don't write so much that you lose engagement
Ask for repetition if needed — say "Could you repeat that question?" (they will re-read it verbatim)	Don't guess at a question you didn't fully hear
Pause briefly before answering — 3–5 seconds of silence is professional, not awkward	Don't rush into an answer before organizing your thoughts
Use the STAR format for behavioral questions (Situation, Task, Action, Result)	Don't ramble without structure — they're filling in a matrix
Say the FJD keywords naturally — "requirements documentation," "peer review," "agency IT standards"	Don't use slang, profanity, or overly casual language
Keep answers to 2 minutes max unless they ask for more detail	Don't monologue for 5+ minutes — panels have a fixed schedule
If you don't know the answer, say: "I haven't worked with that specific technology, but here's how I'd approach learning it — [describe method]"	Don't bluff or make up answers — the technical SME will catch it
Reference your demo project when answering technical questions — "In the compliance tracking app I built..."	Don't give only theoretical answers when you have working code to reference

Body Language

Do	Don't
Sit up straight, lean slightly forward (shows engagement)	Don't slouch or lean back (signals disinterest)
Maintain open posture — hands on the table or in your lap	Don't cross your arms (defensive signal)
Nod and acknowledge when panelists speak	Don't look at your phone, watch, or the door
Smile naturally when appropriate	Don't force a constant grin (it reads as nervous)

After the Interview

Do	Don't
Thank each panelist by name as you leave — "Thank you, [Name], I appreciate your time"	Don't just walk out without thanking them
Ask about next steps: "What does the timeline look like from here?"	Don't ask "Did I get the job?"
Send a thank-you email within 24 hours to the HR contact (not the panelists directly — state panels often prohibit post-interview contact)	Don't email or call panelists directly unless they gave you permission
In your thank-you, reference one specific thing discussed — "I enjoyed discussing the agency's approach to JAD sessions"	Don't send a generic template that could go to any company
Be patient — state timelines are 4–6 weeks, sometimes longer	Don't follow up aggressively every few days
If they request references or background check documents, respond the same day	Don't delay on administrative requests — speed signals enthusiasm

Thank-You Email Template

Subject: Thank You — Programmer IV Interview (Position 00055221)

Dear [HR Contact Name],

Thank you for the opportunity to interview for the Programmer IV position in the Information Resources Division. I enjoyed learning about the team's work and the agency's approach to [specific topic discussed — e.g., "modernizing compliance reporting systems"].

The role's combination of technical leadership, JAD facilitation, and application development aligns strongly with my experience, and I'm enthusiastic about the opportunity to contribute to TCEQ's mission of protecting Texas's natural resources and public health.

Please don't hesitate to reach out if you need any additional information.

Best regards, [Your Name] [Phone] | [Email]

17. System Design Questions

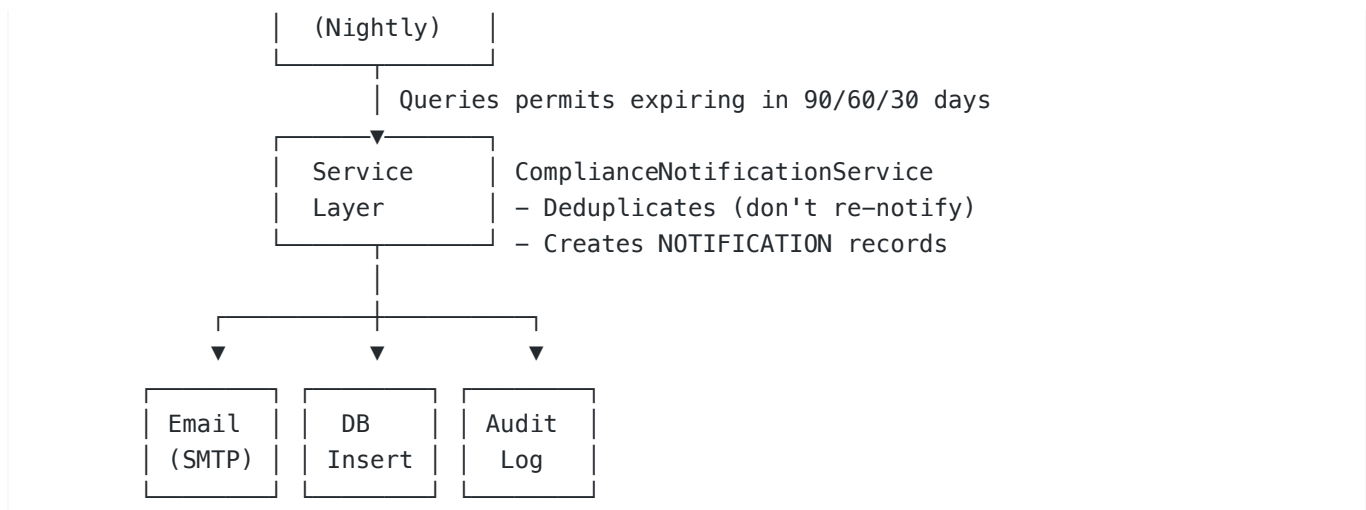
State agency interviews lean toward **practical architecture** over whiteboard system design. But for a Programmer IV, expect "design a subsystem" prompts that test your ability to think in layers.

SD-1: "Design a permit renewal notification system."

Your framework:

Scheduler

(Spring @Scheduled or Oracle DBMS_SCHEDULER)



Key points to hit:

- **Data model:** NOTIFICATIONS (id, permit_id FK, type, sent_date, status)
- **Idempotency:** Check NOT EXISTS (SELECT 1 FROM notifications WHERE permit_id = ? AND type = '30_DAY')
- **Error handling:** If SMTP fails, mark status as FAILED , retry next run
- **Monitoring:** Dashboard widget showing "X permits expiring this month"
- **Change management:** Feature branch → peer review → staging → QA/QC → production with rollback

SD-2: "How would you modernize a monolithic ColdFusion application?"

Answer framework:

Phase	Action	Risk Mitigation
1. Assess	Inventory all CFM pages, rank by business criticality and change frequency	Don't touch stable, rarely-changed pages first
2. Strangler Fig	Put a reverse proxy in front; route specific URLs to new Java/Spring services	Both old and new share the same Oracle datasource
3. Extract services	Move highest-churn business logic into Spring REST APIs; CFM pages call the API	ColdFusion becomes a thin presentation layer
4. Replace UI	One module at a time, replace CFM templates with Thymeleaf or JavaScript SPA	Run in parallel until customer acceptance testing confirms parity
5. Decommission	Remove ColdFusion pages once Java equivalents are fully validated	Keep CFM in source control for historical reference

Key principle: "Never big-bang rewrite. Always incremental with rollback at each phase."

SD-3: "Design a compliance dashboard that serves 500+ concurrent internal users."

Answer:

- **Caching layer:** Cache dashboard aggregates (total facilities, open violations) with a 5-minute TTL — these don't change second-by-second

- **Database:** Materialized views or pre-computed summary tables updated by a scheduled job, not real-time COUNT(*) on every page load
 - **Connection pooling:** HikariCP with pool size tuned to Oracle's max sessions
 - **API design:** Separate endpoints for summary stats vs. detail drill-downs — don't over-fetch
 - **CDN/static:** Serve JS/CSS/images from a CDN or reverse proxy cache
 - **Monitoring:** Slow query log, connection pool saturation alerts, response time SLAs
-

SD-4: "Design an API versioning strategy for agency web services."

Answer:

- **URL-based versioning:** `/api/v1/facilities` , `/api/v2/facilities` — clearest for consumers
 - **Deprecation policy:** Announce v2, support v1 for 12 months minimum (government consumers move slowly), add `Sunset` and `Deprecation` headers to v1 responses
 - **Backwards compatibility rules:** Only add fields (never remove or rename), new required params get defaults, response envelope stays stable
 - **Documentation:** OpenAPI/Swagger spec auto-generated and published per version
 - **Testing:** Contract tests that validate v1 consumers still work against the v2 codebase
-

SD-5: "How would you design a secure file upload system for environmental compliance documents?"

Answer:

- **Validation:** Whitelist file types (PDF, XLSX, CSV only), check MIME type server-side (not just extension), enforce max file size (50MB)
 - **Storage:** Don't store in the database — use filesystem or blob storage with a DB reference
 - **Virus scanning:** ClamAV or agency-approved scanner before persisting
 - **Access control:** Files tagged with `facility_id`; only authorized users can download their facility's documents
 - **Audit trail:** Log every upload/download with user, timestamp, IP, and file hash
 - **Encryption:** At-rest encryption on the storage tier; TLS in transit
-

SD-6: "A critical query is causing deadlocks during the annual emissions inventory period. Diagnose and fix."

Answer:

1. **Identify:** Check Oracle's `V$LOCK` and `DBA_BLOCKERS` views, or `ALTER SYSTEM DUMP` the deadlock graph
 2. **Root cause:** Usually two transactions locking rows in opposite order — Transaction A locks row 1 then wants row 2; Transaction B locks row 2 then wants row 1
 3. **Fix options:**
 - Ensure all transactions lock resources in the **same order** (alphabetical by table, ascending by PK)
 - Reduce transaction scope — commit more frequently
 - Add `SELECT ... FOR UPDATE NOWAIT` so transactions fail fast instead of waiting
 - Move read-heavy dashboard queries to a **read replica** or use `SET TRANSACTION READ ONLY`
 4. **Prevention:** Add deadlock detection to monitoring; alert on lock wait times > 10 seconds
-

18. Algorithm & Data Structure Questions

TCEQ won't ask you to invert a binary tree, but they may give you **practical coding/logic problems**. These are contextual LeetCode-style questions framed for enterprise IT.

Arrays & Strings

#	Question	Approach	Complexity
1	"Given a list of facility EPA IDs, find duplicates."	Use a <code>HashSet</code> — add each ID, if <code>add()</code> returns <code>false</code> it's a duplicate.	$O(n)$ time, $O(n)$ space
2	"Reverse a string without using <code>StringBuilder.reverse()</code> ."	Two-pointer approach: swap chars at <code>i</code> and <code>len-1-i</code> , move inward.	$O(n)$ time, $O(1)$ space
3	"Given a sorted array of inspection dates, find the first overdue one (before today)."	Binary search: compare midpoint to <code>LocalDate.now()</code> , narrow left/right.	$O(\log n)$
4	"Remove duplicate characters from a string while preserving order."	<code>LinkedHashSet<Character></code> — iterate and add; order is preserved, duplicates ignored.	$O(n)$

HashMaps & Sets

#	Question	Approach	Complexity
5	"Count violations by severity from a list of <code>Violation</code> objects."	<code>Map<String, Integer></code> with <code>merge()</code> or <code>Collectors.groupingBy(Violation::getSeverity, Collectors.counting())</code> .	$O(n)$
6	"Find the first non-repeating character in a string."	First pass: build frequency map. Second pass: find first char with <code>count == 1</code> .	$O(n)$
7	"Given two lists of permit numbers, find the intersection."	Add <code>list1</code> to a <code>HashSet</code> , iterate <code>list2</code> and check <code>contains()</code> .	$O(n + m)$

Sorting & Searching

#	Question	Approach	Comp
8	"Sort facilities by compliance score descending, then by name ascending for ties."	<code>Comparator.comparingInt(Facility::getScore).reversed().thenComparing(Facility::getName)</code>	$O(n \log n)$

9	"Merge two sorted lists of inspection records by date."	Two-pointer merge (like merge sort's merge step).	$O(n +$
---	---	---	---------

SQL as Algorithm

#	Question	How It Maps
10	"Find the top 3 counties with the most open violations."	<code>SELECT county, COUNT(*) AS ct FROM violations v JOIN facilities f ON v.facility_id = f.id WHERE v.status = 'OPEN' GROUP BY county ORDER BY ct DESC FETCH FIRST 3 ROWS ONLY;</code>
11	"Find facilities that have permits but no inspections."	<code>SELECT f.* FROM facilities f WHERE EXISTS (SELECT 1 FROM permits p WHERE p.facility_id = f.id) AND NOT EXISTS (SELECT 1 FROM inspections i WHERE i.facility_id = f.id);</code>
12	"Calculate a running total of penalty amounts by date."	Window function: <code>SUM(penalty_amount) OVER (ORDER BY violation_date ROWS UNBOUNDED PRECEDING)</code>

If They Give You a Whiteboard Problem

Strategy:

1. **Clarify inputs/outputs** — "What type is the input? Can it be null? How large?"
2. **Talk through brute force first** — "The naive approach is $O(n^2)$..."
3. **Optimize** — "We can use a HashMap to bring this to $O(n)$..."
4. **Code cleanly** — name variables descriptively, handle edge cases
5. **Test verbally** — "Let me trace through with this example input..."

19. Advanced Java Questions

Concurrency & Multithreading

#	Question	Key Points
1	"What is the difference between <code>synchronized</code> and <code>ReentrantLock</code> ?"	<code>synchronized</code> : implicit lock, auto-released, simpler. <code>ReentrantLock</code> : explicit lock/unlock, supports <code>tryLock()</code> with timeout, interruptible, fairness policy. Use <code>ReentrantLock</code> when you need timeout or try-lock semantics.
2	"What is a race condition? How do you prevent one?"	Two threads modifying shared state simultaneously → unpredictable results. Prevent with: <code>synchronized</code> , <code>Atomic</code> classes (<code>AtomicInteger</code> , <code>AtomicReference</code>), <code>ConcurrentHashMap</code> , or immutable objects. In Spring: stateless services + <code>@Transactional</code> isolation.
3	"Explain <code>volatile</code> keyword."	Guarantees visibility: reads/writes go directly to main memory, not thread-local cache. Does NOT guarantee atomicity — use <code>AtomicInteger</code> for read-modify-write operations.

4	"What is a thread pool? Why use <code>ExecutorService</code> instead of <code>new Thread()</code> ?"	Thread creation is expensive. <code>ExecutorService</code> reuses threads from a pool, controls concurrency, and provides <code>Future</code> for results. In server apps: prevents unbounded thread creation under load. <code>Executors.newFixedThreadPool(10)</code> for bounded parallelism.
5	"What is a <code>CompletableFuture</code> ?"	Composable async operations. Chain with <code>thenApply()</code> , <code>thenCompose()</code> , <code>thenCombine()</code> . Handles errors with <code>exceptionally()</code> . Modern replacement for raw threads when you need async API calls or parallel service invocations.

Generics & Type System

#	Question	Key Points
6	"What is type erasure?"	Java generics are compile-time only. At runtime, <code>List<String></code> and <code>List<Integer></code> are both just <code>List</code> . Can't do <code>new T()</code> or <code>instanceof T</code> at runtime. Implications: bridge methods, unchecked cast warnings.
7	"Explain <code>?</code> extends <code>T</code> vs <code>?</code> super <code>T</code> (PECS)."	<code>?</code> extends <code>T</code> (Producer): read FROM a collection (e.g., <code>List<? extends Violation></code> — you can get <code>Violations</code> out). <code>?</code> super <code>T</code> (Consumer): write INTO a collection (e.g., <code>List<? super Violation></code> — you can put <code>Violations</code> in). PECS: Producer Extends, Consumer Super.

Spring Boot Internals

#	Question	Key Points
8	"What is Spring's <code>@Transactional</code> and how does it work under the hood?"	Spring creates a proxy around the bean. When you call a <code>@Transactional</code> method, the proxy opens a transaction, delegates to the real method, and commits on success / rolls back on <code>RuntimeException</code> . Important: calling <code>@Transactional</code> from within the same class bypasses the proxy — the transaction won't apply.
9	"What is the difference between <code>@Component</code> , <code>@Service</code> , <code>@Repository</code> , and <code>@Controller</code> ?"	All are specializations of <code>@Component</code> (Spring manages the bean lifecycle). Semantic differences: <code>@Repository</code> adds persistence exception translation, <code>@Controller</code> enables MVC request mapping, <code>@Service</code> is a marker for business logic. Use the right one for clarity.
10	"How does Spring dependency injection resolve ambiguity?"	If two beans implement the same interface: <code>@Primary</code> marks the default, <code>@Qualifier("name")</code> selects a specific one, constructor injection with the parameter name matching the bean name.

Memory & Performance

#	Question	Key Points
11	"What is a memory leak in Java? Give an example."	Objects that are still referenced but never used: growing static <code>List</code> , unclosed streams/connections, long-lived <code>HttpSession</code> with large attributes. Detect with heap dumps (<code>jmap</code>) and analyzers (Eclipse MAT, VisualVM). In Hibernate: the first-level cache holds all loaded entities for the session — clear it for batch processing.
12	"What is the difference between <code>==</code> and <code>.equals()</code> in Java?"	<code>==</code> compares references (same object in memory). <code>.equals()</code> compares values (if overridden). <code>String</code> pool makes <code>==</code> sometimes work for strings, but always use <code>.equals()</code> . For enums, <code>==</code> is safe.

20. Web Architecture Questions

CORS (Cross-Origin Resource Sharing)

#	Question	Key Points
1	"What is CORS and why does it exist?"	Browser same-origin policy blocks requests to different domains. CORS is a server-side header protocol that explicitly allows cross-origin requests. Without it, a JavaScript frontend on <code>app.tceq.texas.gov</code> couldn't call an API on <code>api.tceq.texas.gov</code> .
2	"How do you configure CORS in Spring Boot?"	<code>@CrossOrigin</code> on controller, or globally with <code>WebMvcConfigurer.addCorsMappings()</code> . Set allowed origins, methods, headers. In production: never use <code>*</code> for origins — whitelist specific domains.
3	"What is a CORS preflight request?"	For non-simple requests (PUT/DELETE, custom headers), the browser sends an <code>OPTIONS</code> request first. Server responds with <code>Access-Control-Allow-*</code> headers. If the server doesn't respond correctly, the browser blocks the actual request.

Backwards Compatibility

#	Question	Key Points
4	"How do you maintain backwards compatibility when changing an API?"	(1) Add, never remove fields in responses; (2) New required request params get defaults; (3) Version the API URL (<code>/v1/</code> , <code>/v2/</code>); (4) Deprecation headers + migration window; (5) Consumer-driven contract tests that validate old clients still work.
5	"How do you maintain backwards compatibility when changing a database schema?"	(1) Additive schema changes only — new columns are nullable or have defaults; (2) Deploy schema before code; (3) Don't rename columns — add new, migrate data, deprecate old; (4) Versioned migrations with rollback scripts; (5) Both old and new app versions must work against the schema simultaneously.
6	"How do you handle backwards compatibility in web service message formats (XML/JSON)?"	(1) Ignore unknown fields on deserialization (<code>@JsonIgnoreProperties(ignoreUnknown = true)</code>); (2) Add optional fields, never remove required ones; (3) Use schema versioning for XML (namespace-based).

HTTP Fundamentals

#	Question	Key Points
7	"Explain GET vs POST vs PUT vs PATCH vs DELETE."	<code>GET</code> : read (idempotent, cacheable). <code>POST</code> : create (not idempotent). <code>PUT</code> : full replace (idempotent). <code>PATCH</code> : partial update (not idempotent). <code>DELETE</code> : remove (idempotent). Idempotent = safe to retry without side effects.
8	"What are HTTP status codes? Name the key ranges."	<code>2xx</code> : success (200 OK, 201 Created, 204 No Content). <code>3xx</code> : redirect (301/302). <code>4xx</code> : client error (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found). <code>5xx</code> : server error (500 Internal, 502 Bad Gateway, 503 Unavailable). My demo returns 404 for missing facilities via <code>ResponseEntity.notFound()</code> .
9	"What is the difference between 401 and 403?"	401 Unauthorized : "I don't know who you are" (authentication missing/invalid). 403 Forbidden : "I know who you are, but you don't have permission" (authorization failure). Common confusion — 401 should really be called "Unauthenticated."

Caching

#	Question	Key Points
10	"What are the different HTTP caching mechanisms?"	<code>Cache-Control</code> header (max-age, no-cache, no-store). <code>ETag</code> (hash-based validation — server returns 304 Not Modified if unchanged). <code>Last-Modified</code> / <code>If-Modified-Since</code> (date-based). Browser cache vs. CDN cache vs. server-side cache (Redis, in-memory).
11	"When should you NOT cache?"	User-specific data (dashboards with permissions), frequently-changing data (real-time compliance status updates), financial transactions, anything with security implications. Set <code>Cache-Control: no-store, no-cache, must-revalidate</code> .

Sessions & Authentication

#	Question	Key Points
12	"How do sessions work in a web application?"	Server creates a session ID, sends it as a cookie. Client sends the cookie on every request. Server maps session ID → stored data (user info, permissions). Stateful — requires sticky sessions or a shared session store (Redis) for load balancing.
13	"Cookies vs JWT — trade-offs?"	Cookies: server-side session state, simple, built-in browser handling, vulnerable to CSRF. JWT: stateless (token contains claims), no server session, scalable, but can't be revoked easily (until expiry), larger payload. Government apps typically use server-side sessions + CSRF tokens for security.

Security Headers & Web Hardening

#	Question	Key Points
14	"What security headers should every web application set?"	<code>Content-Security-Policy</code> (CSP): prevents XSS by whitelisting script sources. <code>X-Content-Type-Options: nosniff</code> : prevents MIME sniffing. <code>X-Frame-Options: DENY</code> : prevents clickjacking. <code>Strict-Transport-Security</code> (HSTS): forces HTTPS. <code>X-XSS-Protection: 1; mode=block</code> : legacy XSS filter. Spring Security enables most by default.
15	"What is CSRF and how do you prevent it?"	Cross-Site Request Forgery: malicious site tricks your browser into sending a request with your session cookie. Prevent with: CSRF tokens (Spring Security includes by default), <code>SameSite</code> cookie attribute, checking <code>Referer</code> / <code>Origin</code> headers. My ColdFusion forms should include a hidden CSRF token field.